# ORGNZ-09: Enterprise Data Organization Patterns

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 9 of 10 | **Created:** January 2026 | **Last Updated:** 10/14/2026

## Overview

This notebook brings together buckets, segments, and security context into comprehensive enterprise patterns. Learn how to combine these mechanisms for effective data governance at scale.

## Prerequisites

| Requirement | Details |
|---|---|
| **Dynatrace Environment** | SaaS environment with Grail enabled |
| **Permissions** | `storage:bucket-definitions:read`, `storage:logs:read`, `storage:filter-segments:read` |
| **Knowledge** | Completed ORGNZ-08 (Grail Segments) |
| **Data** | At least 1 hour of log and entity data |

## Table of Contents


1. Combining All Three Mechanisms
2. Enterprise Pattern 1: Line of Business Isolation

3. Enterprise Pattern 2: Environment-Based Tiering
4. Enterprise Pattern 3: Multi-Cloud Organization
5. Enterprise Pattern 4: Kubernetes-Native Organization
6. Decision Framework
7. Implementation Checklist
8. Auditing Your Organization
9. Best Practices Summary
10. Series Summary

---

## Learning Objectives

By the end of this notebook, you will:
- Combine buckets, segments, and security context effectively
- Design enterprise-scale data organization architectures
- Implement common organizational patterns
- Plan data governance strategies

## Combining All Three Mechanisms

For comprehensive data governance, use all three mechanisms:

| Mechanism | Purpose | When to Use |
|---|---|---|
| **Buckets** | Physical storage, retention | Different retention or access needs (cost attribution uses `dt.cost.*` fields) |
| **Segments** | Logical filtering, views | Team-focused data views, cross-signal filtering |
| **Security Context** | Access enforcement | Fine-grained security requirements |

### Complete Architecture Example

```
Finance Team Requirements:

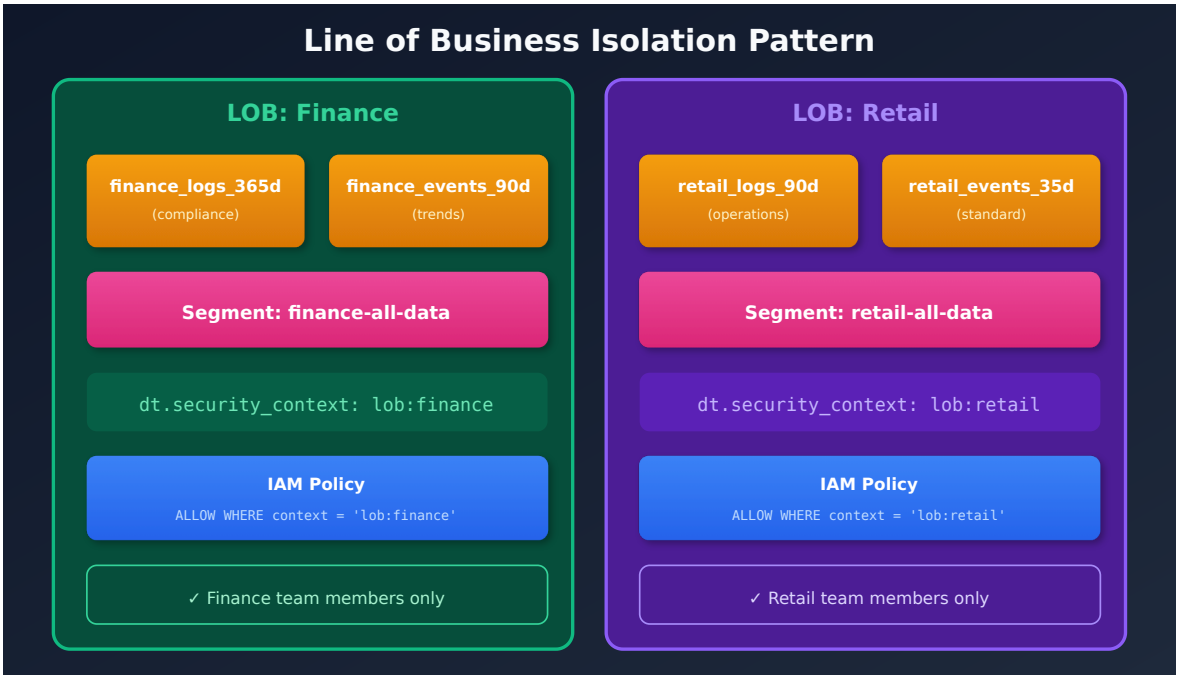
Bucket: finance_logs_365d
  - 1 year retention for compliance
  - Cost attributed with dt.cost.costcenter set at the source (not by the
bucket)

Segment: finance-team-view
  - Filtered view of finance data
  - Includes related services and hosts

Security Context: lob:finance
  - IAM policy: ALLOW WHERE security_context = 'lob:finance'
  - Only finance team members can access
```

## Enterprise Pattern 1: Line of Business Isolation

Complete isolation by business unit:




### Implementation

#### Step 1: Create buckets

```
finance_logs_365d (logs, 365 days)
finance_events_90d (events, 90 days)
retail_logs_90d (logs, 90 days)
```

**Step 2: Assign buckets in OpenPipeline**

In the log pipeline's **Storage** stage, add one **Bucket assignment** processor per LOB; the first matching processor wins (see **ORGNZ-03 § Routing Data to Buckets**):

| Matching condition (DQL) | Bucket |
|---|---|
| `matchesValue(dt.host_group.id, "finance-*")` | `finance_logs_365d` |
| `matchesValue(dt.host_group.id, "retail-*")` | `retail_logs_90d` |

**Step 3: Set security context**

Set `lob:finance` / `lob:retail` at the source first — a `dt.security_context=lob:finance` host tag on the finance hosts, or a fixed value per host group in the central primary Grail tag configuration (see **ORGNZ-06 §3**). Where the value has to be computed in the pipeline, use **Set security context** processors in the pipeline's Permission stage, which *"Sets the proper record-level access via dt.security_context by either copying it from a field, setting it as a static string, or a static array that allows multiple values."*

| Matching condition (DQL) | Security context (static string) |
|---|---|
| `matchesValue(dt.host_group.id, "finance-*")` | `lob:finance` |
| `matchesValue(dt.host_group.id, "retail-*")` | `lob:retail` |

> **Corrected 09/24/2026.** Earlier versions of this notebook showed `type: route` and `type: security-context` YAML processors keyed on `host.group`. Neither processor type exists in that form, and `host.group` is not a Grail field — use `dt.host_group.id`.
>
> **Sources:** Processing in OpenPipeline (DT docs), DQL matcher in OpenPipeline (DT docs).

**Step 4: Create IAM policies**

```
// Finance policy
ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH "finance_";
ALLOW storage:logs:read, storage:events:read WHERE storage:dt.security_context
= "lob:finance";
```

# Enterprise Pattern 2: Environment-Based Tiering

Different access levels for production vs non-production:

```
Production:
├── Buckets: prod_logs, prod_events, prod_spans
├── Security Context: env:production
├── Policy: ALLOW WHERE storage:dt.security_context = 'env:production'
├── Access: Restricted to production support team
└── Audit: Full audit logging enabled

Non-Production:
├── Buckets: default_logs, default_metrics, default_spans
├── Security Context: env:dev, env:staging, env:qa
├── Policy: ALLOW WHERE storage:dt.security_context MATCH ('env:dev*',
'env:staging*')
├── Access: Broader developer access
└── Audit: Reduced audit requirements
```

## Tiered Access Groups

| Group | Production Access | Non-Prod Access |
|---|---|---|
| SRE Team | Full | Full |
| Production Support | Read-only | None |
| Developers | None | Full |
| QA | None | Read-only |

# Enterprise Pattern 3: Multi-Cloud Organization

Organize by cloud provider and account:

```
AWS:
├── Bucket: aws_logs_35d
├── Security Context: cloud:aws/&lt;account-id&gt;
└── Policy: ALLOW WHERE storage:aws.account.id = '123456789012'

Azure:
├── Bucket: azure_logs_35d
├── Security Context: cloud:azure/&lt;subscription&gt;
└── Policy: ALLOW WHERE storage:azure.subscription = 'sub-id'

GCP:
├── Bucket: gcp_logs_35d
├── Security Context: cloud:gcp/&lt;project&gt;
└── Policy: ALLOW WHERE storage:gcp.project.id = 'my-project'
```

# Enterprise Pattern 4: Kubernetes-Native Organization

Leverage Kubernetes constructs for organization:

```
Cluster: main-cluster
├── Namespace: production
|   ├── Security Context: k8s:main-cluster/production
|   └── Policy: ALLOW WHERE storage:k8s.namespace.name = 'production'
|
├── Namespace: staging
|   ├── Security Context: k8s:main-cluster/staging
|   └── Policy: ALLOW WHERE storage:k8s.namespace.name = 'staging'
|
└── Namespace: development
    ├── Security Context: k8s:main-cluster/development
    └── Policy: ALLOW WHERE storage:k8s.namespace.name = 'development'
```

## Cross-Namespace Segment

One application deployed to several namespaces, as one segment. Build it in the Segments app with a single include:

| Include (data object) | Filter |
| --- | --- |
| Data (all types) | `k8s.workload.name = "checkout"` |

Segment conditions take only `=` and `in()` — *"Only operators `=`, and `in()` are supported."* — with `*` wildcards inside the value, so write `=` and double quotes, not DQL's `==`. One `Data (all types)` include replaces separate `logs` and `spans` includes. Check coverage first: `k8s.workload.name` is a `stable` semantic-dictionary field, but on the validation tenant on 10/14/2026 it was on 41% of logs in an hour, and records without it fall outside the segment (adapt the coverage audit in ORGNZ-10 §9).

> **Sources:** Segment limits (DT docs) — as quoted, Supported data types in segments (DT docs) — *"We recommend constructing segments with a single Data (all types) include block when possible."*
>
> **Dictionary:** `k8s.workload.name` (`stable`), read 10/14/2026.

## Decision Framework

Use this framework to choose your organization strategy:



# Implementation Checklist

## Phase 1: Planning

- [ ] Identified retention requirements
- [ ] Mapped organizational structure (teams, LOBs, cost centers)
- [ ] Defined access control requirements
- [ ] Planned bucket naming convention
- [ ] Designed security context schema

## Phase 2: Infrastructure

- [ ] Created custom buckets
- [ ] Configured OpenPipeline bucket assignment (Storage stage)
- [ ] Configured OpenPipeline security context processors
- [ ] Verified data flow to correct buckets

## Phase 3: Access Control

- [ ] Created IAM policies
- [ ] Assigned policies to groups
- [ ] Tested access with sample users
- [ ] Documented all policies

## Phase 4: Views

- [ ] Created segments for team views
- [ ] Tested segment filters
- [ ] Documented segment purposes

## Phase 5: Governance

- [ ] Planned access review process
- [ ] Configured audit logging
- [ ] Created runbook for access management

# Auditing Your Organization

## DQL: Environment Audit

Measure security context adoption across all ingested logs:

```
// Audit security context coverage — percentage of logs with
dt.security_context assigned
fetch logs, from:-1h
| summarize
    total = count(),
    withContext = countIf(isNotNull(dt.security_context)),
    withoutContext = countIf(isNull(dt.security_context))
| fieldsAdd coverage = round(toDouble(withContext) / toDouble(total) * 100,
decimals: 2)
```

# Best Practices Summary

| Area | Best Practice |
|---|---|
| **Buckets** | Target 1–3 TB/day per log bucket, a dedicated bucket per application above 250 GB/day (ORGNZ-02); use consistent naming |
| **Segments** | Test rules with DQL before deploying |
| **Security Context** | Use hierarchical encoding for flexibility |
| **IAM Policies** | Assign to groups, not individuals |
| **Documentation** | Document all policies and contexts |
| **Reviews** | Schedule regular access reviews |

# Series Summary

The ORGNZ series covered:

| Notebook | Topic | Key Takeaway |
|---|---|---|
| ORGNZ-01 | Introduction | Why organize data; three pillars |
| ORGNZ-02 | Grail Buckets | Bucket fundamentals and limits |
| ORGNZ-03 | Bucket Strategy | Naming, retention, design patterns |
| ORGNZ-04 | Permissions Overview | Permission levels and policy structure |
| ORGNZ-05 | Bucket-Level Access | IAM policies for bucket isolation |
| ORGNZ-06 | Security Context | Fine-grained access with dt.security_context |
| ORGNZ-07 | Advanced Permissions | Record and field-level patterns |
| ORGNZ-08 | Grail Segments | Dynamic data organization |
| ORGNZ-09 | Enterprise Patterns | Combined approaches at scale |
| ORGNZ-10 | Advanced Segment Definitions | Filter syntax, enrichment, variables, troubleshooting |

## References


- Organize data
- Permissions in Grail
- Advanced permission setup
- Enhance data management with Grail
- Set up Cost Allocation (DT docs)

*This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*